

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
Engenharia de Computação

ANA LAURA KOEFENDER FÜHR
LUÍS GUSTAVO ERICH DUARTE

**ESTATÍSTICAS TEMPORAIS GERADAS A PARTIR DE DIFERENTES
TIPOS DE ÁRVORES BINÁRIAS**

Porto Alegre
2025

ESTATÍSTICAS TEMPORAIS GERADAS A PARTIR DE DIFERENTES TIPOS DE ÁRVORES BINÁRIAS

Trabalho apresentado para as aulas de Estrutura de Dados, da Universidade Federal do Rio Grande do Sul, em 2025.

Professores: Dennis Giovani Balreira e Viviane Pereira Moreira

Porto Alegre
2025

SUMÁRIO

1. INTRODUÇÃO.....	4
2. METODOLOGIA E FUNÇÕES UTILIZADAS.....	5
3. ANÁLISE DE DESEMPENHO.....	8
4. CONCLUSÃO.....	12
5. REFERÊNCIAS.....	13

1. INTRODUÇÃO

As árvores são estruturas de dados extremamente importantes para o desenvolvimento de diversos projetos na computação, com inúmeras aplicações no nosso dia a dia, já que possuem uma relação hierárquica entre seus nodos. Uma das possíveis aplicações será demonstrada durante a execução deste trabalho, que é a extração de dados de um dataset fornecido pela plataforma Steam, para informar o tempo médio total entre os usuários para terminar uma lista específica de jogos.

Além disso, faremos a comparação de eficiência e tempo de resposta de dois tipos específicos de árvores binárias: AVL (árvores balanceadas por altura), que consideram apenas a distância entre os nodos e a raiz, então quanto mais balanceada a estrutura, mais rapidamente acessamos todos os seus nodos; e as ABP (árvores binárias de pesquisa), as quais apresentam uma relação de ordem nos campos de informações dos seus nodos, denominados chaves.

2. METODOLOGIA E FUNÇÕES UTILIZADAS

Para a realização dos códigos, foram feitas pesquisas em livros sobre o tópico, com destaque para “Estrutura de Dados”, de Nina Edelweiss e Renata Galante, repositórios e sites particulares, além da documentação fornecida pelos professores da disciplina. Ao longo desta seção iremos detalhar melhor as funções utilizadas.

Primeiramente, devemos destacar que algumas funções pertencem a bibliotecas prontas, como `stdio`, `stdlib`, `string`, `time` e `locale`. Já a `main` consiste na inicialização de variáveis, configuração do cronômetro e das acentuações e então as operações que utilizam as funções especificadas no arquivo `.c`. O código receberá a base de dados “`arq_dataset`”, a lista de jogos escolhidos “`arq_lista`” e também uma seleção entre os dois tipos de árvores “`abp|avl`”, e com base nisso, inicia o processo de carregamento e armazenamento da tabela de jogos e suas horas médias nas diferentes estruturas, conforme optado pelo usuário, e esse passo será cronometrado.

Posteriormente o algoritmo fará uma busca apenas dos jogos listados pelo usuário no arquivo passado por parâmetro na função `main`, também contabilizando o tempo gasto. Diante disso, o programa encerra suas funções e retorna na tela os resultados finais, cria o arquivo de saída e limpa a memória, destruindo as árvores.

```
printf("\n--- Resultados Finais ---\n");
printf("Tipo de Arvore: %s\n", tipoArvore);
printf("Tempo Total de Carregamento: %.6f s\n", time_load);
printf("Tempo Total de Busca: %.6f s\n", time_search);
printf("Altura da Árvore: %d\n", alturaArvore);
printf("Total de Horas a Jogar: %.2f horas\n", totalHoras);
printf("Total de Comparacao: %d comparacoes\n", comp);
printf("Total de Rotações: %d rotações\n", nm_rotacao);
printf("Total de Nodos: %d nodos\n", nm_nodos);
```

Na subdivisão `.h` do projeto, que se refere ao cabeçalho das funções e declaração das structs, sendo elas uma responsável por cada nodo - baseado no

modelo fornecido pelos professores e utilizado durante as aulas práticas da disciplina -, contendo os ponteiros para a esquerda e direita, o fator de balanceamento, a fim de facilitar as rotações, no caso da AVL, além dos próprios dados do tipo “JogoInfo”, criado anteriormente e parte de outra struct, com o nome e as horas.

```
typedef struct {  
    char nome[150];  
    float horas;  
} JogoInfo;  
  
typedef struct NodoArvore {  
    JogoInfo info;           // dados do jogo  
  
    struct NodoArvore *esq;  
  
    struct NodoArvore *dir;  
  
    int fb;                  // adicionamos a info de fator de  
                             balanceamento, para facilitar as rotações AVL  
} pNodoA;
```

```
// inicializa a árvore e retorna null  
pNodoA* inicializa();  
  
// destroi a árvore (libera toda a memória)  
pNodoA* destroi(pNodoA* a);  
  
// busca um jogo pelo nome, retorna a info do jogo se achar e null se  
// não achar  
JogoInfo* busca(pNodoA* a, char* nome, int *comp);  
  
// funções da ABP  
  
// Insere um jogo numa ABP  
pNodoA* insereABP(pNodoA* a, JogoInfo info);
```

```

// funções da AVL

//insere um jogo numa AVL
pNodoA* insereAVL(pNodoA* a, JogoInfo info, int *cresceu, int
*nm_rotacao);

// funções de rotação
pNodoA* rotacao_direita(pNodoA* p);
pNodoA* rotacao_esquerda(pNodoA* p);
pNodoA* rotacao_dupla_direita(pNodoA* p);
pNodoA* rotacao_dupla_esquerda(pNodoA* p);

// casos de desbalanceamento
pNodoA* Caso1(pNodoA* a, int *cresceu);
pNodoA* Caso2(pNodoA* a, int *cresceu);

//retorna a altura da árvore
int Altura(pNodoA* a);

```

Por fim, o .c contém o detalhamento de todas as funções utilizadas na main e com os seus respectivos cabeçalhos na .h, nele utilizamos majoritariamente os códigos da professora Viviane Pereira Moreira, como é o caso da inicialização, destruição, busca - que foi baseada no código fornecido no enunciado deste trabalho como pNodoA* consulta -, inserção e rotação.

3. ANÁLISE DE DESEMPENHO

A partir dos dados gerados por alguns casos de teste, fomos capazes de comparar o desempenho das árvores em diferentes categorias, abaixo podemos observar uma planilha com alguns deles, os quais foram destacados em cores para os piores - mais tempo gasto, logo menor desempenho, em vermelho.

	Total de Nodos	Total de Rotações	Tempo de Carregamento em segundos	Tempo Total de Busca em segundos	Altura da Árvore	Total de Comparações
AVL - todos os jogos	3597	1806	0,005	0	14	2124
ABP - todos os jogos	3597	0	0,004	0	28	3081

AVL - 4 jogos	7	1	0	0,001	4	15
ABP - 4 jogos	7	0	0	0,003	4	13

AVL - 2 jogos	3	1	0	0,003	2	4
ABP - 2 jogos	3	0	0	0,001	3	2

AVL - 4 jogos	6	0	0	0,002	3	11
ABP - 4 jogos	6	0	0	0,001	6	19

Teste 1 - contabilizou todos os jogos, ou seja, 3597 nodos, e resultou no tempo total máximo: 2589 horas;

Teste 2 - contabilizou 7 jogos (listados à esquerda) e buscou 4 deles (à direita), ato que necessita de um total de 375 horas para “zerá-los”.

- call of duty, 30
- Free Fire, 200
- lol, 45
- God of War, 70
- Zombie Night Terror, 370
- Ace Attorney, 60
- Yuppie Psycho, 50
- Free Fire
- lol
- Ace Attorney
- God of War

Teste 3 - contabilizou 3 jogos (listados à esquerda) e buscou 2 deles (à direita), ato que necessita de um total de 60 horas para “zerá-lo

- God of War, 70
- Zombie Night Terror, 370
- Ace Attorney, 60

- Ace Attorney
- God of War

Teste 4 - contabilizou 6 jogos (listados à esquerda) e buscou 4 deles (à direita), ato que necessita de um total de 130 horas para “zerá-los

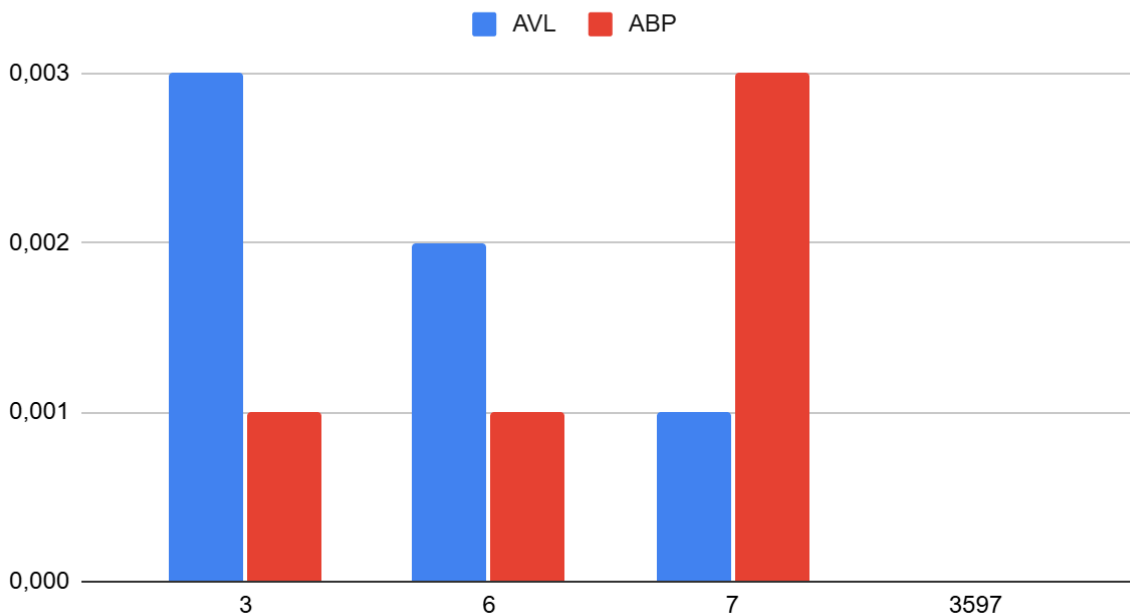
- Zombie Night Terror, 370
- Ace Attorney, 60
- Free Fire, 46
- GTA, 30
- Minecraft, 75
- Magic Sword, 40

- Ace Attorney
- Free Fire
- Magic Sword
- GTA

Com a coleta de dados, foi possível organizarmos os diferentes tópicos em gráficos do número de nodos em função do tempo de carregamento, tempo de busca, altura da árvore e número de comparações. O número de rotações foi descartado para essa parcela da pesquisa, pois apenas as AVLs realizam essa operação.

Gráfico 1:

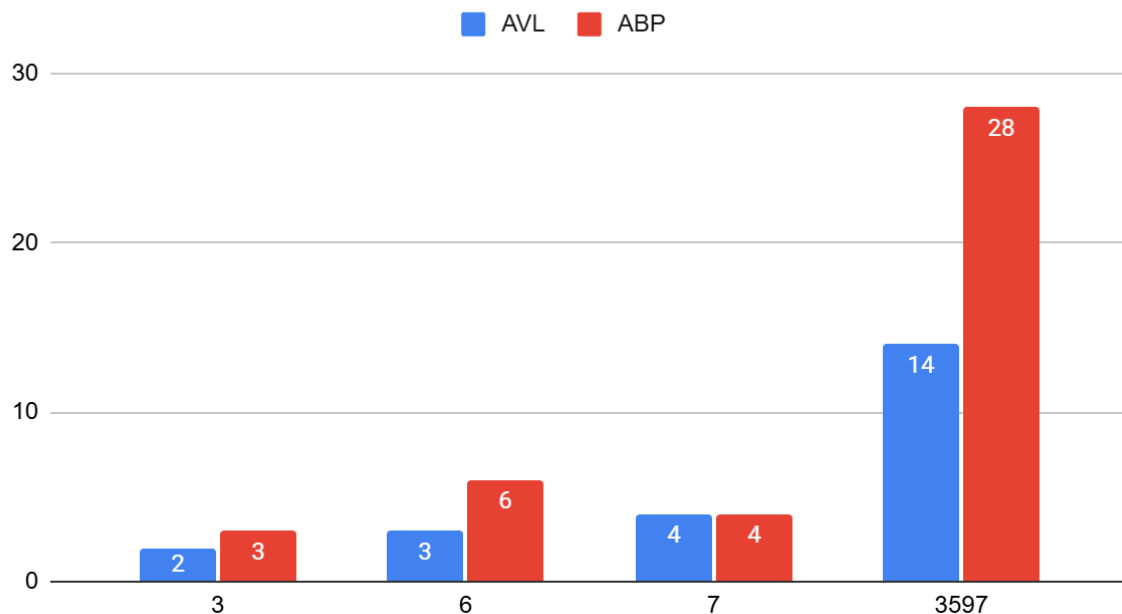
Tempo Total de Busca(s) X Número de Nodos



É possível perceber que, quando analisamos o tempo de busca, a AVL tende a mostrar-se mais eficiente.

Gráfico 2:

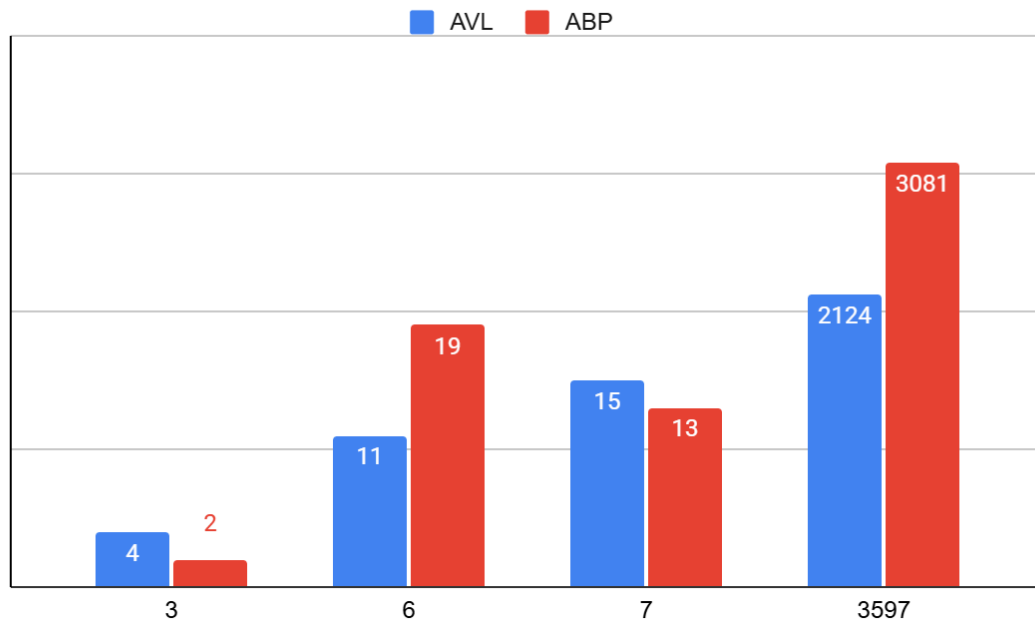
Altura da Árvore X Número de Nodos



Conforme o segundo gráfico, fomos capazes de constatar que as AVLs tendem a formar árvores com uma menor altura, resultado totalmente esperado, uma vez que possuem a capacidade de rotacionar, garantindo um percurso menor entre a raiz e as folhas, e consequentemente, uma altura menor. Enquanto as ABPs preocupam-se apenas em organizar os dados conforme as chaves.

Gráfico 3:

Número de Comparações X Número de Nodos



Nesse caso fica ainda mais claro que as possibilidades de escolha entre jogos listados e escolhidos são quase que infinitas, e por isso, a forma que os dados podem ser inseridos nas estruturas de dados também, resultando diferentes padrões e combinações que podem ou não ser mais eficientes.

3. CONCLUSÃO

Diante disso podemos perceber que, apesar de tanto as ABPs quanto as AVLs tratarem-se de árvores binárias, elas utilizam algoritmos diferentes para as mesmas funções, e por isso, cada uma delas mostrou-se mais eficiente para, pelo menos, uma das situações simuladas nos casos de teste.

Apesar disso, as árvores balanceadas por altura (AVL) acabam se destacando positivamente na maioria das situações, já que não dependem de uma ordem de inserção favorável para tornar o caminho entre a raiz e o nodo procurado mais curto, e sim, realizam as rotações necessárias para diminuir o custo independentemente de quais operações serão feitas em seguida, logo não é vantajoso optarmos pelas árvores binárias de pesquisa, apesar da suas utilidades em casos específicos.

4. REFERÊNCIAS

EDELWEISS, Nina; GALANTE, Renata. **Estrutura de Dados**. 1ª Edição.
Porto Alegre: Bookman, 2009.